TypeScript: 50টি Scenario-Based MCQ with Answers

তোমার দেওয়া TypeScript Complete Beginner Guide PPT থেকে beginner level থেকে NestJS-এর প্রস্তুতি পর্যন্ত গুরুত্বপূর্ণ topic নিয়ে ৫০টি scenario-based MCQ তৈরি করেছি। প্রতিটি প্রশ্নে বাস্তব coding situation দেওয়া আছে, সঙ্গে সঠিক উত্তর ও সহজ বাংলা ব্যাখ্যা।

TypeScript.pptx

প্রশ্নগুলো এই topic-গুলো কভার করবে:

- Basic types, type annotation ও type inference
- Arrays, objects ও interfaces
- Optional properties, functions ও return types
- Union types, literal types ও `type` vs `interface`
- Classes, constructors, access modifiers ও inheritance
- `async/await`, `Promise<T>` ও generics
- API response, common mistakes ও NestJS

Section 1: Basic Types, Variables & Arrays

1. বয়স সংরক্ষণ

একজন student-এর বয়স `25`। তুমি চাও variable-এ শুধু number রাখা যাবে। কোনটি সঠিক?

- A. `let age: string = 25;`
- B. `let age: number = 25;`
- C. `let age: boolean = 25;`
- D. `let age = true;`

সঠিক উত্তর: B

ব্যাখ্যা: `number` type ব্যবহার করলে variable-এ number value রাখতে হবে।

2. ভুল type assignment

একটি variable-এর type `number`। নিচের কোডে কী হবে?

```
let age: number = 20;
age = "twenty";
```

- A. কোডটি সঠিক
- B. `age` string হয়ে যাবে
- C. TypeScript type error দেখাবে
- D. `age` মুছে যাবে

সঠিক উত্তর: C

ব্যাখ্যা: `age`-এর type number, কিন্তু `"twenty"` একটি string।

3. Type inference

```
let name = "Rahim";
name = 100;
```

কী সমস্যা হবে?

- A. কোনো সমস্যা হবে না
- B. TypeScript inferred `string` type-এর সঙ্গে assignment-টি মিলবে না
- C. `name` automatically number হবে
- D. `let` কাজ করবে না

সঠিক উত্তর: B

ব্যাখ্যা: প্রথম value দেখে TypeScript `name`-কে string হিসেবে infer করে।

4. Student-এর নামের তালিকা

একটি array-তে শুধু student-এর নাম রাখতে হবে। কোনটি সঠিক?

- A. `let names: number[] = ["Rahim", "Karim"];`
- B. `let names: string[] = ["Rahim", "Karim"];`
- C. `let names: boolean[] = ["Rahim"];`
- D. `let names: number = ["Rahim"];`

সঠিক উত্তর: B

ব্যাখ্যা: `string[]` মানে array-এর element-গুলো string হবে।

5. Array-তে ভুল data

```
let marks: number[] = [80, 90];
marks.push("100");
```

কী হবে?

- A. সঠিকভাবে কাজ করবে
- B. TypeScript type error দেখাবে
- C. Array string array হয়ে যাবে
- D. `marks` null হবে

সঠিক উত্তর: B

ব্যাখ্যা: `marks`-এ number রাখা যাবে; `"100"` string।

6. Object থেকে নাম নেওয়া

```
const student = {
    id: 10,
    name: "Hamim"
};

console.log(student.name);
```

Output কী?

- A. `10`
- B. `Hamim`
- C. `name`
- D. `undefined`

সঠিক উত্তর: B

ব্যাখ্যা: `student.name` object-এর `name` property-এর value দেয়।

7. Type annotation

একটি variable-এর type স্পষ্টভাবে number নির্ধারণ করতে চাও। কোন syntax সঠিক?

- A. `let marks = number: 80;`
- B. `let marks: number = 80;`
- C. `number let marks = 80;`
- D. `let number marks = 80;`

সঠিক উত্তর: B

ব্যাখ্যা: TypeScript-এ variable-এর নামের পরে colon দিয়ে type লেখা হয়।

8. String array-তে নতুন নাম

```
let names: string[] = ["A", "B"];
names.push("C");
```

Array-টি কী হবে?

- A. `["A", "B", "C"]`
- B. `["A", "B"]`
- C. `[1, 2, 3]`
- D. Type error

সঠিক উত্তর: A

ব্যাখ্যা: `"C"` string, তাই `string[]` array-তে যোগ করা বৈধ।

Section 2: Interface, Optional Properties & Functions

9. Student-এর structure

```
interface Student {
    id: number;
    name: string;
}
```

এই interface-এর উদ্দেশ্য কী?

- A. Student object-এর structure নির্ধারণ করা
- B. Database তৈরি করা
- C. Automatically student object তৈরি করা
- D. Function চালানো

সঠিক উত্তর: A

ব্যাখ্যা: Interface বলে দেয় object-এ কোন property এবং type থাকতে হবে।

10. Required property বাদ পড়েছে

```
interface User {
    id: number;
    name: string;
}

const user: User = {
    id: 1
};
```

সমস্যা কী?

- A. `id` number হওয়া যাবে না
- B. `name` required কিন্তু দেওয়া হয়নি
- C. Interface ব্যবহার করা যাবে না
- D. `const` ভুল

সঠিক উত্তর: B

ব্যাখ্যা: `name` optional নয়, তাই object-এ সেটি দিতে হবে।

11. Optional CGPA

কিছু student-এর CGPA আছে, কিছু student-এর নেই। কোন declaration সঠিক?

- A. `cgpa: number`
- B. `cgpa?: number`
- C. `cgpa!: number`
- D. `cgpa: string[]`

সঠিক উত্তর: B

ব্যাখ্যা: `?` property-টিকে optional করে।

12. Optional property ব্যবহার

```
interface Student {
    id: number;
    name: string;
    cgpa?: number;
}
```

```
const s: Student = {
    id: 2,
    name: "Rahim"
};
```

কী হবে?

- A. Type error হবে
- B. বৈধ Student object হবে
- C. `id` optional হবে
- D. `name` বাদ যাবে

সঠিক উত্তর: B

ব্যাখ্যা: `cgpa` optional হওয়ায় সেটি না দিলেও object বৈধ।

13. Function-এর parameter type

দুটি number যোগ করার function লিখতে হবে। কোনটি সঠিক?

- A. `function add(a: string, b: string): number`
- B. `function add(a: number, b: number): number`
- C. `function add(a: boolean, b: boolean): string`
- D. `function add(a, b): boolean`

সঠিক উত্তর: B

ব্যাখ্যা: `a` ও `b` number, এবং return type-ও number।

14. Function-এর output

```
function add(a: number, b: number): number {
    return a + b;
}

console.log(add(10, 20));
```

Output কী?

- A. `1020`
- B. `30`
- C. `10`
- D. `Error`

সঠিক উত্তর: B

ব্যাখ্যা: দুটি number যোগ করলে `30` পাওয়া যায়।

15. শুধু print করার function

একটি function শুধু নাম print করবে, কোনো value return করবে না। উপযুক্ত return type কোনটি?

- A. `string`
- B. `number`
- C. `void`
- D. `Student[]`

সঠিক উত্তর: C

ব্যাখ্যা: `void` এমন function-এর return type বোঝাতে ব্যবহৃত হয়, যেটি কোনো অর্থপূর্ণ value return করে না।

16. Interface-সহ function

```
interface Student {
    id: number;
    name: string;
}

function displayStudent(student: Student): void {
    console.log(student.name);
}
```

Function-টি কী ধরনের argument আশা করে?

- A. যেকোনো string
- B. একটি number
- C. `Student` structure অনুসরণ করা object
- D. শুধু boolean

সঠিক উত্তর: C

ব্যাখ্যা: `student: Student` বলে argument-টিকে Student interface অনুসরণ করতে হবে।

17. ভুল property type

```
interface Student {
    id: number;
```

```
    name: string;
    cgpa: number;
}

const s: Student = {
    id: 1,
    name: "Rahim",
    cgpa: "3.80"
};
```

ভুলটি কোথায়?

- A. `id`-এর type ভুল
- B. `name`-এর type ভুল
- C. `cgpa` string, কিন্তু number প্রয়োজন
- D. কোনো ভুল নেই

সঠিক উত্তর: C

ব্যাখ্যা: `"3.80"` quotation marks-সহ string, number নয়।

Section 3: Union, Literal Types & Type Alias

18. Student ID

Student ID কখনো number, কখনো string হতে পারে। কোনটি ব্যবহার করবে?

- A. `number & string`
- B. `number | string`
- C. `number[]`
- D. `boolean`

সঠিক উত্তর: B

ব্যাখ্যা: Union type `|` একাধিক অনুমোদিত type-এর মধ্যে যেকোনো একটি গ্রহণ করতে পারে।

19. Union type-এ boolean

```
let id: number | string;
id = true;
```

কী হবে?

- A. বৈধ হবে
- B. Type error হবে

- C. `id` automatically boolean হবে
- D. `id` array হবে

সঠিক উত্তর: B

ব্যাখ্যা: এখানে শুধু number অথবা string অনুমোদিত।

20. Status নির্ধারণ

একটি account-এর status শুধু `"active"` অথবা `"inactive"` হতে পারবে। কোনটি সঠিক?

- A. `type Status = string;`
- B. `type Status = "active" | "inactive";`
- C. `type Status = number | boolean;`
- D. `type Status = string[];`

সঠিক উত্তর: B

ব্যাখ্যা: Literal union type নির্দিষ্ট কিছু value-তে অনুমতি সীমাবদ্ধ করে।

21. অবৈধ status

```
type Status = "active" | "inactive";
let status: Status = "deleted";
```

কী হবে?

- A. বৈধ হবে
- B. Type error হবে
- C. `status` number হবে
- D. `status` optional হবে

সঠিক উত্তর: B

ব্যাখ্যা: `"deleted"` অনুমোদিত literal-এর মধ্যে নেই।

22. Type alias

একটি reusable ID type তৈরি করতে হবে, যা number অথবা string হবে। কোনটি সঠিক?

- A. `type ID = number | string;`
- B. `interface ID = number | string;`
- C. `class ID = number | string;`

- D. `let ID: interface;`

সঠিক উত্তর: A

ব্যাখ্যা: `type` দিয়ে union type-এর একটি নাম দেওয়া যায়।

23. Interface বনাম type

কোন বক্তব্যটি PPT-এর সঙ্গে মেলে?

- A. `interface` শুধু number-এর জন্য
- B. `type` দিয়ে union ও literal type তৈরি করা যায়
- C. `type` এবং `interface` কখনো object structure বর্ণনা করতে পারে না
- D. TypeScript-এ শুধু `interface` ব্যবহার করা যায়

সঠিক উত্তর: B

ব্যাখ্যা: `type` union, literal এবং type composition-এর জন্য বিশেষভাবে flexible।

24. Intersection type চেনা

`A & B`-এর অর্থ কী?

- A. A অথবা B
- B. শুধু A
- C. এমন type, যা A এবং B উভয়ের শর্ত পূরণ করে
- D. কোনো type নেই

সঠিক উত্তর: C

ব্যাখ্যা: `&` হলো intersection; `|` হলো union।

Section 4: Classes, Constructors & Inheritance

25. Student class

```
class Student {
    name: string;
    age: number;
}
```

`class Student` কী বর্ণনা করে?

- A. Object তৈরি করার blueprint
- B. শুধু একটি array
- C. শুধু একটি interface
- D. একটি Promise

সঠিক উত্তর: A

ব্যাখ্যা: Class data ও behavior-সহ object তৈরি করার blueprint।

26. Constructor

```
class Student {
    constructor(public name: string) {}
}

const s = new Student("Rahim");
console.log(s.name);
```

Output কী?

- A. `Student`
- B. `Rahim`
- C. `name`
- D. `undefined`

সঠিক উত্তর: B

ব্যাখ্যা: Constructor-এ `"Rahim"` দেওয়া হয়েছে এবং `public name` property তৈরি হয়েছে।

27. Public property

কোন access modifier property-কে class-এর বাইরে থেকেও access করতে দেয়?

- A. `private`
- B. `protected`
- C. `public`
- D. `async`

সঠিক উত্তর: C

ব্যাখ্যা: `public` property সাধারণভাবে class-এর বাইরে থেকেও access করা যায়।

28. Private balance

```
class BankAccount {
```

```
    private balance: number = 1000;
}

const account = new BankAccount();
console.log(account.balance);
```

কী সমস্যা হবে?

- A. `balance` public হয়ে যাবে
- B. Class-এর বাইরে private property access করার কারণে TypeScript error হবে
- C. Output হবে `1000`
- D. `balance` array হবে

সঠিক উত্তর: B

ব্যাখ্যা: `private` member সরাসরি class-এর বাইরে access করা যায় না।

29. Protected property

কোন modifier parent class এবং তার subclass-এ member access করার জন্য ব্যবহৃত হয়, কিন্তু সাধারণ বাইরের access সীমিত রাখে?

- A. `public`
- B. `protected`
- C. `private`
- D. `string`

সঠিক উত্তর: B

ব্যাখ্যা: `protected` member class ও subclass-এর ভেতরে ব্যবহার করা যায়।

30. Private data access

Bank account-এর private balance নিরাপদভাবে ব্যবহার করার জন্য কোন design ভালো?

- A. Balance-কে সরাসরি public করে দেওয়া
- B. Public method দিয়ে balance দেখা বা পরিবর্তন করা
- C. সব property মুছে ফেলা
- D. Balance-কে array করা

সঠিক উত্তর: B

ব্যাখ্যা: Public method-এর মাধ্যমে private data নিয়ন্ত্রিতভাবে access করা যায়।

31. Inheritance

```
class Person {
    introduce(): void {
        console.log("Hello");
    }
}

class Student extends Person {}
```

`Student` কী করতে পারবে?

- A. `introduce()` ব্যবহার করতে পারবে
- B. `Person` মুছে ফেলবে
- C. `Person`-এর কোনো method পাবে না
- D. শুধু string return করবে

সঠিক উত্তর: A

ব্যাখ্যা: `extends` দিয়ে Student, Person-এর method inherit করে।

32. `super()` কেন ব্যবহার করা হয়?

```
class Person {
    constructor(public name: string) {}
}

class Student extends Person {
    constructor(name: string, public studentId: number) {
        super(name);
    }
}
```

এখানে `super(name)`-এর কাজ কী?

- A. Student class মুছে ফেলা
- B. Parent class-এর constructor call করা
- C. `studentId`-কে string বানানো
- D. Promise তৈরি করা

সঠিক উত্তর: B

ব্যাখ্যা: `super(name)` parent `Person`-এর constructor call করে inherited অংশ initialize করে।

33. Class-এর member access

একটি class-এর `private password` property বাইরের code থেকে সরাসরি পড়তে চাইলে কী হবে?

- A. সবসময় অনুমোদিত হবে
- B. TypeScript access error হবে
- C. Property public হয়ে যাবে
- D. Property automatically optional হবে

সঠিক উত্তর: B

ব্যাখ্যা: `private` member-এর access class-এর ভেতরে সীমাবদ্ধ।

Section 5: Async/Await & Promises

34. Asynchronous name

```
async function getName(): Promise<string> {
    return "Hamim";
}
```

Function-টির return type কী?

- A. `string`
- B. `number`
- C. `Promise<string>`
- D. `Student[]`

সঠিক উত্তর: C

ব্যাখ্যা: `async` function Promise return করে; এখানে ফলাফলটি string।

35. Await ব্যবহার

```
async function getName(): Promise<string> {
    return "Hamim";
}

async function main(): Promise<void> {
    const name = await getName();
    console.log(name);
}
```

`await getName()` কী করে?

- A. Promise-এর ফলাফলের জন্য অপেক্ষা করে, তারপর execution এগোয়

- B. Function মুছে দেয়
- C. সবসময় number return করে
- D. `getName()`-কে array বানায়

সঠিক উত্তর: A

ব্যাখ্যা: `await` asynchronous ফলাফল পাওয়ার পর পরবর্তী কাজ চালাতে সাহায্য করে।

36. Promise-এর Student

```
interface Student {
    id: number;
    name: string;
}

async function getStudent(): Promise<Student> {
    return { id: 1, name: "Hamim" };
}
```

`Promise<Student>` বলতে কী বোঝায়?

- A. Student-এর array
- B. পরে একটি Student object পাওয়া যাবে
- C. একটি string পাওয়া যাবে
- D. একটি boolean পাওয়া যাবে

সঠিক উত্তর: B

ব্যাখ্যা: Promise resolve হলে একটি Student object পাওয়া যাবে।

37. একাধিক Student

```
async function getStudents(): Promise<Student[]> {
    return [
        { id: 1, name: "Hamim" },
        { id: 2, name: "Rahim" }
    ];
}
```

এখানে `Promise<Student[]>` কেন ব্যবহার করা হয়েছে?

- A. কারণ একটি number return হবে
- B. কারণ পরে Student object-এর array পাওয়া যাবে
- C. কারণ function string নেয়
- D. কারণ array নিষিদ্ধ

সঠিক উত্তর: B

ব্যাখ্যা: `Student[]` হলো Student-এর array; `Promise` বোঝায় asynchronous ফলাফল।

38. Async function-এ ভুল return type

```
async function getAge(): Promise<number> {
    return "25";
}
```

সমস্যা কী?

- A. `"25"` string, কিন্তু `number` প্রত্যাশিত
- B. `async` ব্যবহার করা যাবে না
- C. `Promise<number>` মানে string
- D. `return` ব্যবহার করা যাবে না

সঠিক উত্তর: A

ব্যাখ্যা: Function-এর resolved result number হওয়া দরকার, string নয়।

39. Backend-এ asynchronous কাজ

PPT অনুযায়ী, কোন কাজগুলোতে `async/await` প্রায়ই ব্যবহৃত হয়?

- A. Database queries
- B. HTTP requests
- C. File operations ও external APIs
- D. উপরের সবগুলো

সঠিক উত্তর: D

ব্যাখ্যা: এই ধরনের কাজ প্রায়ই asynchronous হয়।

Section 6: Generics & API Responses

40. Generic function

```
function getData<T>(data: T): T {
    return data;
}
```

এখানে `T` কী?

- A. সবসময় string
- B. একটি type placeholder
- C. একটি database
- D. একটি constructor

সঠিক উত্তর: B

ব্যাখ্যা: Generic type placeholder-এর মাধ্যমে function বিভিন্ন type-এর data নিয়ে কাজ করতে পারে।

41. Generic-এ string

```
function getData<T>(data: T): T {
    return data;
}

const name = getData("Hamim");
```

`name`-এর inferred type কী?

- A. `number`
- B. `boolean`
- C. `string`
- D. `Student[]`

সঠিক উত্তর: C

ব্যাখ্যা: Argument string হওয়ায় TypeScript সাধারণত `T`-কে string হিসেবে infer করে।

42. Generic-এ number

```
const age = getData(25);
```

`age`-এর inferred type কী?

- A. `string`
- B. `number`
- C. `boolean`
- D. `void`

সঠিক উত্তর: B

ব্যাখ্যা: `25` একটি number; TypeScript `T` infer করে number হিসেবে।

43. Generic-এর সুবিধা

একটি function-কে string, number এবং boolean—বিভিন্ন type-এর data নিয়ে কাজ করাতে চাও। কোন approach ভালো?

- A. সবকিছু `any` করে দেওয়া
- B. Generic `<T>` ব্যবহার করা
- C. প্রতিবার code মুছে নতুন function বানানো
- D. সব value-কে string করা

সঠিক উত্তর: B

ব্যাখ্যা: Generic reusable code-কে type-safe রাখতে সাহায্য করে।

44. Generic API response

```
interface ApiResponse<T> {
    success: boolean;
    data: T;
}
```

`T`-এর কাজ কী?

- A. `success` property মুছে দেওয়া
- B. `data`-এর type বিভিন্ন প্রয়োজন অনুযায়ী নির্ধারণ করা
- C. সব data number করা
- D. শুধু Student object অনুমোদন করা

সঠিক উত্তর: B

ব্যাখ্যা: একই response structure বিভিন্ন data type-এর সঙ্গে ব্যবহার করা যায়।

45. Student API response

```
interface Student {
    id: number;
    name: string;
}

const response: ApiResponse<Student> = {
    success: true,
    data: { id: 1, name: "Hamim" }
};
```

`response.data`-এর type কী?

- A. `string`
- B. `number`
- C. `Student`
- D. `Student[]`

সঠিক উত্তর: C

ব্যাখ্যা: `ApiResponse<Student>`-এ generic argument হলো `Student`।

46. Student list API response

তুমি API থেকে অনেক Student-এর list পাঠাতে চাও। কোন type ব্যবহার করবে?

- A. `ApiResponse<string>`
- B. `ApiResponse<Student[]>`
- C. `ApiResponse<boolean>`
- D. `ApiResponse<number>`

সঠিক উত্তর: B

ব্যাখ্যা: একাধিক Student হলে `Student[]` ব্যবহার করবে।

Section 7: Common Mistakes & NestJS Scenarios

47. `any` ব্যবহারের ঝুঁকি

```
let data: any = "hello";
data = 100;
data = true;
```

কোন বক্তব্যটি সঠিক?

- A. `any` শুধু string অনুমোদন করে
- B. `any` বিভিন্ন type-এর value গ্রহণ করে, তবে type safety কমায়
- C. `any` সবসময় error দেয়
- D. `any` হলো array type

সঠিক উত্তর: B

ব্যাখ্যা: `any` ব্যবহার করলে TypeScript-এর type-checking সুবিধার অনেকটাই হারাতে পারো।

48. NestJS Controller

```
@Controller("students")
```

```
export class StudentController {
    constructor(
        private studentService: StudentService
    ) {}
}
```

এই code-এ `StudentController`-এর মূল ভূমিকা কী?

- A. Students-সম্পর্কিত requests handle করার controller হিসেবে কাজ করা
- B. সব student-এর নাম hardcode করা
- C. TypeScript compile করা
- D. একটি generic type তৈরি করা

সঠিক উত্তর: A

ব্যাখ্যা: `@Controller("students")` students route-এর সঙ্গে controller-কে যুক্ত করে।

49. NestJS Service

```
@Injectable()
export class StudentService {
    async findAll(): Promise<Student[]> {
        // ...
    }
}
```

`findAll()`-এর ঘোষিত return type কী?

- A. `Promise<string>`
- B. `Student`
- C. `Promise<Student[]>`
- D. `boolean`

সঠিক উত্তর: C

ব্যাখ্যা: Method-টি asynchronous এবং এর declared result হলো Student-এর array।

50. বাস্তব Backend Scenario

তুমি একটি Student API তৈরি করছ। API-তে একাধিক Student থাকবে, প্রত্যেকের `id` ও `name` বাধ্যতামূলক, কিন্তু `cgpa` optional। `getStudents()` asynchronous হবে।

কোন solution-টি সবচেয়ে উপযুক্ত?

- A.

```
interface Student {
    id: number;
    name: string;
    cgpa?: number;
}

async function getStudents(): Promise<Student[]> {
    return [
        { id: 1, name: "Rahim" },
        { id: 2, name: "Hamim", cgpa: 3.9 }
    ];
}
```

- B.

```
interface Student {
    id: string;
    name: number;
    cgpa: boolean;
}
```

- C.

```
function getStudents(): number {
    return "Rahim";
}
```

- D.

```
let students: boolean = [
    { id: 1, name: "Rahim" }
];
```

সঠিক উত্তর: A

ব্যাখ্যা: এখানে interface, required properties, optional property, object array, `async` এবং `Promise<Student[]>`—সবকিছু সঠিকভাবে ব্যবহার করা হয়েছে।

Final Answer Key

দ্রুত রিভিশনের জন্য সব সঠিক উত্তর একসঙ্গে:

প্রশ্ন	উত্তর	প্রশ্ন	উত্তর
1	B	26	B
2	C	27	C
3	B	28	B
4	B	29	B
5	B	30	B

6	B	31	A
7	B	32	B
8	A	33	B
9	A	34	C
10	B	35	A
11	B	36	B
12	B	37	B
13	B	38	A
14	B	39	D
15	C	40	B
16	C	41	C
17	C	42	B
18	B	43	B
19	B	44	B
20	B	45	C
21	B	46	B
22	A	47	B
23	B	48	A
24	C	49	C
25	A	50	A

পরামর্শ: প্রথমে উত্তর না দেখে ৫০টি MCQ solve করো। তারপর Answer Key দিয়ে মিলিয়ে ভুল topic-গুলো আবার revise করো। বিশেষ করে `interface`, `Promise<T>`, generics এবং `async/await` ভালোভাবে বুঝে নিলে NestJS Backend শুরু করা সহজ হবে।